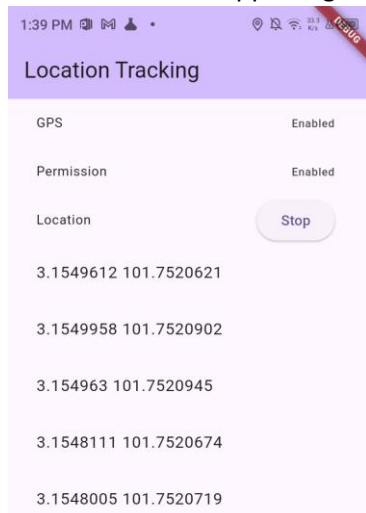


Practical 13: Location Tracking

1. We will create an app using location tracking using device GPS or wireless network.



2. Create a new mobile app using the Application template.
3. Insert the following dependencies:
location
permission_handler
4. Open the android/app/src/main/AndroidManifest.xml file. Insert the following permissions.

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
  <!-- TODO 1 - Insert android permissions -->
  <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
  <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />

  <application ...>
```

The ACCESS_FINE_LOCATION includes GPS data in reporting user location.

The ACCESS_COARSE_LOCATION includes data from the most battery-efficient non-GPS provider available (for example, the network).

5. Open the main.dart file. Import the following libraries:

```
//TODO 2 - Import libraries
import 'dart:async';
import 'dart:developer';
import 'package:permission_handler/permission_handler.dart' as handler;
import 'package:location/location.dart';
```
6. Within the _MyHomePageState, insert the following:

```
//TODO 3 - Declare local variables
bool _permissionGranted = false;
bool _gpsEnabled = false;
bool _trackingEnabled = false;
late StreamSubscription _subscription;
final Location _location = Location();
final List<LocationData> _locationList = [];
```

7. Within the `_MyHomePageState`, create two async methods to check location and GPS permission status:


```
//TODO 4 - Check is permission granted
Future<bool> isPermissionGranted() async {
  return await handler.Permission.locationWhenInUse.isGranted;
}

//TODO 5 - Check is GPS enabled
Future<bool> isGpsEnabled() async {
  return await handler.Permission.location.serviceStatus.isEnabled;
}
```
8. Insert the `checkStatus` method, which determines the location and GPS permission status.


```
//TODO 6 - Check permission status
void checkStatus() async {
  bool permissionGranted = await isPermissionGranted();
  bool gpsEnabled = await isGpsEnabled();
  setState(() {
    _permissionGranted = permissionGranted;
    _gpsEnabled = gpsEnabled;
  });
}
```
9. Override the `initState()` method.


```
//TODO 7 - check status on initialisation
@override
void initState() {
  super.initState();
  checkStatus();
}
```
10. Within the `_MyHomePageState`, create two async methods that start or end location tracking:


```
//TODO 8 - start tracking user's location
void startTracking() async {
  if (!(await isGpsEnabled())) {
    return;
  }
  if (!(await isPermissionGranted())) {
    return;
  }
  _subscription = _location.onLocationChanged.listen((event) {
    addLocation(event);
  });
  setState(() {
    _trackingEnabled = true;
  });
}
```

```
//TODO 9 - stop tracking user's location
void stopTracking() {
    _subscription.cancel();
    setState(() {
        _trackingEnabled = false;
    });
    clearLocation();
}
```

11. Override the dispose method. We should stop the location tracking when the app enters the dispose stage.

```
//TODO 10 - stop tracking on dispose
@override
void dispose() {
    stopTracking();
    super.dispose();
}
```

12. Within the `_MyHomePageState`, we may request user permission to enable GPS location service:

```
//TODO 11 - Request for GPS
void requestEnableGps() async {
    if (_gpsEnabled) {
        log("Already open");
    } else {
        bool isGpsActive = await _location.requestService();
        if (!isGpsActive) {
            setState(() {
                _gpsEnabled = false;
            });
            log("User did not turn on GPS");
        } else {
            log("gave permission to the user and opened it");
            setState(() {
                _gpsEnabled = true;
            });
        }
    }
}
```

13. Similarly, we may also request permission to access user's location:

```
//TODO 12 - Request location permission
void requestLocationPermission() async {
    var permissionStatus =
        await handler.Permission.locationWhenInUse.request();

    if (permissionStatus == PermissionStatus.granted) {
        setState(() {
            _permissionGranted = true;
        });
    } else {
        setState(() {
            _permissionGranted = false;
        });
    }
}
```

14. From time to time, location data delivered by location service are inserted to a location list:

```
//TODO 13 - Add a new location record to the location list
void addLocation(LocationData data) {
    setState(() {
        _locationList.insert(0, data);
    });
}
```

15. When user press the Stop button, we shall clear location data from the location list:

```
//TODO 14 - Clear the location list
void clearLocation() {
    setState(() {
        _locationList.clear();
    });
}
```

16. Create a widget to display a location record:

```
//TODO 15 - Create a list tile template
ListTile buildListTile(
    String title,
    Widget? trailing,
) {
    return ListTile(
        dense: true,
        title: Text(title),
        trailing: trailing,
    );
}
```

17. Within the build method, create the main UI:

```
//TODO 16 - Create the UI
body:Padding(
    padding: const EdgeInsets.symmetric(horizontal: 12),
    child: Column(
        mainAxisAlignment: MainAxisAlignment.max,
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
            buildListTile(
                "GPS",
                _gpsEnabled
                ? const Text("Enabled")
                : ElevatedButton(
                    onPressed: () {
                        requestEnableGps();
                    },
                    child: const Text("Enable Gps")),
            ),
        ],
    ),
```

```

        buildListTile(
            "Permission",
            _permissionGranted
                ? const Text("Enabled")
                : ElevatedButton(
                    onPressed: () {
                        requestLocationPermission();
                    },
                    child: const Text("Request Permission")),
        ),
        buildListTile(
            "Location",
            _trackingEnabled
                ? ElevatedButton(
                    onPressed: () {
                        stopTracking();
                    },
                    child: const Text("Stop"))
                : ElevatedButton(
                    onPressed: _gpsEnabled && _permissionGranted
                        ? () {
                            startTracking();
                        }
                        : null,
                    child: const Text("Start")),
        ),
        Expanded(
            child: ListView.builder(
                itemCount: _locationList.length,
                itemBuilder: (context, index) {
                    return ListTile(
                        title: Text(
                            "${_locationList[index].latitude}
                            ${_locationList[index].longitude}"),
                    );
                },
            ),
        ),
    ],
),
),
),

```



TODO: Enhance the app by showing the user current location using the Open Street Map.